

AgriGuard AI

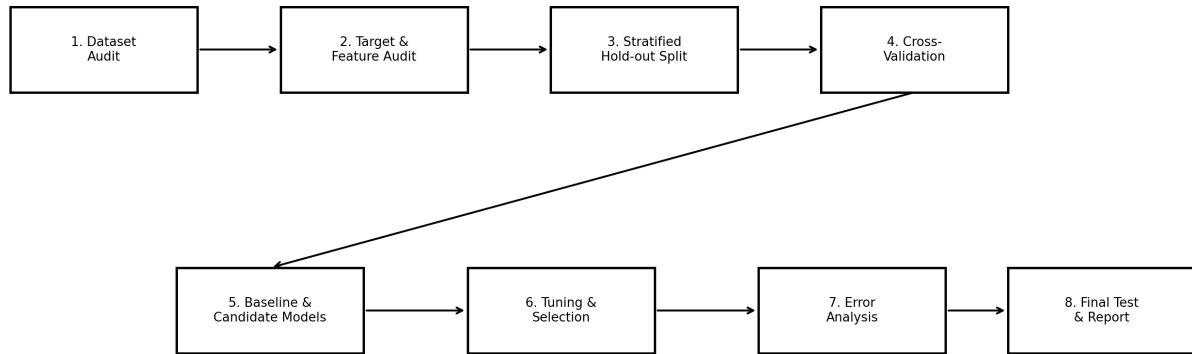
WEEK 3

Experimental Design and Evaluation Metric Formulation

Virtual Machine Learning Implementation Intern

A reproducible evaluation plan for crop disease-risk classification

AgriGuard AI — Week 3 Experimental Design



Integrity rule: development data is used for validation/tuning; the hold-out test set is locked until the final evaluation.

1. Executive Summary

Week 3 establishes a rigorous experimental framework for testing the machine-learning approach proposed in Week 2 of AgriGuard AI. The objective is not simply to calculate a single accuracy value, but to determine whether the model is reliable across classes, stable across validation folds, resistant to common sources of leakage, and sufficiently understandable to support later implementation decisions.

The proposed experiment follows a controlled sequence: dataset audit, target and feature verification, locked hold-out test split, stratified cross-validation on development data, baseline comparison, candidate-model evaluation, hyperparameter tuning, error analysis, and one-time final testing. The primary model-selection measure is macro F1, supported by class-wise recall and precision. Accuracy remains useful as a general indicator, but it is deliberately not treated as the sole success criterion because an imbalanced risk dataset can produce deceptively high accuracy.

The report also responds directly to the previous evaluation feedback requesting stronger evidence and more detailed risk mitigation. The current AgriGuard repository contains project documentation and a data README rather than a verified populated training dataset. Therefore, this document does not manufacture row counts, class percentages, feature ranges or model results. Instead, it defines a concrete evidence-capture procedure so that real values can be inserted once the verified dataset is available. This is important for maintaining scientific credibility and reproducibility.

2. Project Context and Problem Definition

AgriGuard AI is designed as a proposed decision-support system for estimating crop disease risk from agricultural and environmental observations. The Week 2 design considered variables such as temperature, humidity, rainfall, leaf wetness, crop age and previous disease observations. The Week 3 objective is to determine how well a classifier can distinguish the defined risk categories and where its predictions fail.

The practical problem is asymmetric: not all classification mistakes have the same consequence. A false alarm may cause unnecessary inspection or intervention, while a missed high-risk situation may delay attention. Consequently, the experimental design must expose class-specific failures instead of hiding them inside a single aggregate score.

3. Objectives

- Establish a reproducible procedure for evaluating the Week 2 model design.
- Compare a simple baseline with Logistic Regression, Decision Tree, Random Forest and Gradient Boosting.
- Use stratified cross-validation to estimate development-stage generalization and variability.
- Protect the final test set from model selection and repeated experimentation.
- Measure accuracy, precision, recall, F1, macro F1 and confusion-matrix behaviour.
- Perform structured error analysis to identify why important misclassifications occur.
- Document risks, mitigation actions, contingency plans and evidence required to verify each control.
- Produce an experiment log that another person could use to reproduce the evaluation.

4. Research Questions and Hypotheses

Null hypothesis (H0): the candidate models do not provide a meaningful improvement over the baseline under the same experimental protocol.

Alternative hypothesis (H1): at least one candidate model provides a meaningful improvement while maintaining acceptable class-wise recall and reasonable validation stability.

Decision principle: macro F1 is the primary selection metric, but it is interpreted together with class-wise recall, precision, variability and the confusion matrix. No model is selected solely because it produces the largest accuracy value.

5. Dataset Evidence Plan

Concrete dataset evidence was specifically requested in the previous feedback. Since a verified populated AgriGuard dataset is not currently present in the repository, the following audit will be performed before model training. The resulting values will be recorded in the experiment log rather than estimated or invented.

Example of the required evidence format after dataset acquisition: “Dataset shape = [verified value]; target distribution = [verified class counts/percentages]; missingness = [verified values].” Brackets are intentionally left as placeholders until real data are available.

6. Data Quality and Preprocessing Protocol

- Validate column names and data types before any model operation.
- Separate the target from predictive features before preprocessing.
- Remove or investigate duplicate observations according to documented rules.
- Check impossible physical values using domain constraints where available.
- Handle missing numeric values using a method selected only from development data; avoid calculating imputation statistics from the test set.
- Encode categorical variables within the training pipeline so validation information cannot leak into training.
- Scale features only for algorithms that require or benefit from scaling, such as Logistic Regression.
- Keep preprocessing and model fitting inside a single pipeline during cross-validation.
- Record every transformation, excluded field and reason for exclusion.

The preprocessing pipeline is part of the model. Treating preprocessing separately and fitting it once on the entire dataset could allow information from validation or test observations to influence the model indirectly.

7. Experimental Split Strategy

A final hold-out test set will be separated before candidate models are compared. For a classification problem, stratification is preferred where class support permits it. An illustrative planning split is 80% development data and 20% test data; the actual split can be adjusted if the verified dataset size or class support requires a different design.

The development set is the only data used for cross-validation, feature-selection decisions and hyperparameter tuning. The hold-out test set is locked and evaluated once after the final pipeline is selected. If the test set is inspected repeatedly during development, its role changes from an unbiased final estimate to another tuning resource.

If observations are strongly time-dependent, a random split may not reflect deployment conditions. In that case, a time-aware validation design will be considered. If multiple observations come from the same farm, plot or location, grouped validation may also be required to prevent related observations from appearing in both training and validation folds.

8. Cross-Validation Design

The primary development-stage strategy is Stratified K-Fold cross-validation. Five folds are proposed initially. Each fold is used once as validation while the remaining folds are used for training, and stratification attempts to preserve class proportions. The exact number of folds will be reconsidered if the rarest class has insufficient observations.

A model with a slightly higher mean macro F1 but very high fold-to-fold variability should be investigated rather than automatically accepted. Stability is evidence about reliability, not merely an optional statistic.

9. Candidate Algorithms and Experimental Role

The comparison is controlled: all candidate models receive the same target definition, development/test split, fold structure, scoring metrics and documented preprocessing policy.

10. Metric Formulation and Rationale

10.1 Accuracy

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$. It answers the broad question of how many predictions were correct. It is useful for an overall reference but may hide minority-class failure when class frequencies are uneven.

10.2 Precision

Precision = $TP / (TP + FP)$. It indicates how trustworthy positive/class-specific warnings are. High precision is desirable when false alarms create unnecessary inspection or intervention.

10.3 Recall

Recall = $TP / (TP + FN)$. It measures how many true members of a class are detected. In an agricultural early-warning context, recall for an important high-risk class deserves explicit attention because missed cases can be more consequential than extra alerts.

10.4 F1 Score

$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$. F1 balances precision and recall using their harmonic mean. It is useful when both false alarms and missed cases matter.

10.5 Macro F1

Macro F1 calculates F1 for every class and gives each class equal weight before averaging. This is the primary model-selection metric because it prevents a large majority class from dominating the overall score.

10.6 Confusion Matrix

The confusion matrix maps actual classes to predicted classes. It is essential for identifying directional errors, such as high-risk observations repeatedly being classified as medium risk. Raw counts and normalized percentages should both be inspected where appropriate.

10.7 Balanced Accuracy

Balanced accuracy can supplement ordinary accuracy when class frequencies differ. It averages recall across classes and therefore provides another view of whether performance is being dominated by a frequent class.

Metric policy: macro F1 is primary; class-wise recall is a safeguard; precision, accuracy, balanced accuracy and the confusion matrix provide context. Metrics will be calculated on validation/test predictions generated without leakage.

11. Experimental Phases and Controls

12. Error Analysis Framework

Aggregate metrics indicate how much the model is wrong; error analysis investigates why. The following procedure will be applied after each serious candidate has been evaluated:

- Generate a confusion matrix and identify the largest off-diagonal error patterns.
- Calculate per-class precision, recall and F1 to locate weak classes.
- Separate false positives from false negatives for the classes that matter operationally.
- Inspect representative incorrect observations and compare them with correctly classified observations.
- Group errors by available context such as crop type, season, region, temperature band or humidity band.

- Check whether missing values, rare categories or extreme values are associated with errors.
- Check whether similar observations receive inconsistent predictions.
- Form a root-cause hypothesis and test it with a controlled change using development data only.
- Record whether the intervention improved the target metric and what trade-off it created.

For example, if high-risk recall is low, possible causes include insufficient high-risk examples, noisy labels, overlapping class definitions, weak features or an unsuitable decision boundary. The experiment should distinguish these possibilities rather than assuming that changing the algorithm will solve the problem.

13. Detailed Risk, Mitigation and Contingency Plan

14. Model Selection Decision Framework

Model selection will not be based on one number. The following order will be applied:

- First confirm that the model and preprocessing pipeline passed leakage and data-quality checks.
- Compare macro F1 across models using the same cross-validation protocol.
- Inspect class-wise recall, especially for the operationally important risk class.
- Review precision to understand the false-alarm burden.
- Review mean and standard deviation across folds for stability.
- Inspect confusion matrices and major error categories.
- Consider model complexity and interpretability when performance is close.
- Only after selection, evaluate the chosen pipeline on the locked hold-out test set.

If the best-performing model is only marginally better than a simpler model, the simpler model may be preferred for a prototype if it offers materially better interpretability and operational reliability. The decision and evidence will be documented rather than hidden.

15. Hyperparameter Experiment Plan

Parameter search will be deliberately bounded. Excessive tuning on a small dataset can itself produce overfitting to cross-validation. All tuning decisions remain inside the development data.

16. Reproducibility and Experiment Log

Each run should record:

- Dataset filename/source, version or retrieval date and integrity identifier where practical.
- Exact target definition and feature list.
- Train/test split method and random seed.
- Cross-validation type, number of folds and random seed.
- Preprocessing steps and fitted transformations.
- Model name and complete hyperparameters.
- Per-fold metric values and aggregate mean $\pm$ standard deviation.
- Confusion matrix and class-wise metrics.
- Code version/Git commit identifier.
- Reason for every feature exclusion or experimental decision.

The experiment log is a safeguard against selective reporting. If a later result differs from an earlier result, the recorded configuration should make it possible to identify why.

17. Python Implementation Blueprint

LOAD the verified AgriGuard dataset

RECORD source, version, shape, schema and target definition

AUDIT:

missing values
duplicates
class distribution
numeric ranges
categorical levels
potential leakage

CREATE an untouched TEST_SET using stratification when appropriate
DEFINE DEVELOPMENT_SET as the remaining data

BUILD a fold-safe preprocessing pipeline

DEFINE candidate models:

baseline
Logistic Regression
Decision Tree
Random Forest
Gradient Boosting

DEFINE StratifiedKFold with a fixed random seed

FOR each candidate:

combine preprocessing and model in one pipeline
run cross-validation on DEVELOPMENT_SET
record macro F1, recall, precision, accuracy and balanced accuracy
store each fold result
calculate mean and standard deviation

SELECT using macro F1 + class-wise recall + stability + practical complexity

TUNE the selected candidate using DEVELOPMENT_SET only

REFIT the selected pipeline

EVALUATE TEST_SET once

GENERATE confusion matrix and error-analysis report

SAVE metrics, configuration, plots and experiment log

18. Acceptance Criteria

19. Limitations and Scope

- The current project repository does not contain a verified populated AgriGuard training dataset, so this planning report makes no fabricated claims about sample size, class balance or predictive accuracy.
- Cross-validation estimates performance under the sampled data distribution; it cannot guarantee performance in a different season, region or farm environment.
- Metric quality depends on the reliability and consistency of the target labels.
- Environmental and agricultural relationships can change over time, creating distribution shift.
- Model probability outputs should not automatically be interpreted as agronomic certainty.
- The prototype should be treated as decision support and validated by domain experts before any real-world high-impact use.

20. Seven-Day Implementation Timeline

21. Expected Evidence Package

- Dataset audit table containing verified shape, target distribution, missingness and feature summaries.
- Cross-validation results for each candidate model, including fold-level scores.
- Model comparison table with macro F1, recall, precision and accuracy.
- Confusion matrix for the selected model.

- Error-analysis notes with representative failure categories.
- Final hold-out test results, clearly separated from CV results.
- Experiment configuration and Git commit/reference for reproducibility.

22. Conclusion

The Week 3 experimental design provides a disciplined path for evaluating the AgriGuard AI model proposed in Week 2. Rather than treating accuracy as the final answer, the experiment evaluates class balance, recall, precision, macro F1, stability and error patterns. The use of an untouched hold-out test set protects the final estimate, while stratified cross-validation provides a more informative view of development-stage variability.

The plan also strengthens the project in the areas highlighted by the previous evaluation. Concrete dataset evidence is explicitly required and will be measured from the verified data rather than invented. Risk mitigation is expanded into a measurable process: every major risk has a detection signal, impact, prevention/mitigation action, contingency plan and verification method. This makes the experimental plan more than a theoretical description; it becomes an implementation-ready roadmap.

The final goal is not to produce an impressive-looking metric, but to establish whether the proposed model behaves reliably, where it fails, and whether those failures are acceptable for the intended prototype scope. Any real-world performance claim will be made only after the actual dataset has been processed and the complete experiment has been executed.

23. References

- Scikit-learn — Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation
- Scikit-learn — StratifiedKFold documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html
- Scikit-learn — confusion_matrix documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html
- Scikit-learn — classification_report documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html
- Scikit-learn — f1_score documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
- Scikit-learn — recall_score documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html
- Scikit-learn — precision_score documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html
- Food and Agriculture Organization of the United Nations — Plant Production and Protection. <https://www.fao.org/plant-production-protection/about/>
- Food and Agriculture Organization of the United Nations — Pest and Pesticide Management. <https://www.fao.org/pest-and-pesticide-management/about/understanding-the-context/en>

Table 1

ID	Question	Evidence
RQ1	Does the ML pipeline improve on the baseline?	CV and final test metrics
RQ2	Which model provides the best balanced class performance?	Macro F1 + per-class recall
RQ3	Where does the selected model fail?	Confusion matrix + error groups
RQ4	Are results stable across validation folds?	Mean and standard deviation
RQ5	Could data quality or leakage distort the result?	Audit and controlled checks

Table 2

Evidence	Procedure	Why it matters	Output
Rows/columns	Load dataset and inspect shape	Defines sample and feature scale	Dataset shape
Target classes	Count and percentage by class	Detects imbalance	Class table
Missing values	Null count and percentage per feature	Guides cleaning	Missingness table
Numeric ranges	Min, quartiles, median, max	Detects invalid/extreme values	Statistics table
Categorical values	Unique and rare categories	Prevents encoding problems	Category table

Evidence	Procedure	Why it matters	Output
Duplicates	Exact and key duplicate checks	Prevents repeated observations	Duplicate count
Temporal coverage	Date range if available	Checks temporal shift/leakage	Coverage summary
Geographic coverage	Region/location distribution if available	Checks geographic generalization	Coverage summary
Label provenance	Source and definition of target	Measures label reliability	Label notes
Leakage candidates	Check features created after outcome	Prevents artificial performance	Leakage decision log

Table 3

Stage	Action	Control
Fold creation	Stratify by target where appropriate	Preserve class proportions
Preprocessing	Fit only on each training fold	Prevent leakage
Training	Fit model on fold training portion	Same settings across models
Validation	Predict fold validation portion	Never train on validation rows
Scoring	Calculate predefined metrics	Avoid selective metrics
Aggregation	Mean + standard deviation	Show stability

Table 4

Model	Role	Strength	Main Concern	Evaluation Focus
Majority/simple baseline	Reference point	Easy to reproduce	May ignore minority classes	Shows whether ML adds value
Logistic Regression	Interpretable baseline	Simple, fast, explainable	Linear decision boundary	Macro F1 + coefficients
Decision Tree	Nonlinear benchmark	Readable rules	Can overfit	Depth vs CV performance
Random Forest	Primary candidate	Robust nonlinear ensemble	Less transparent than one tree	Macro F1 + recall + stability
Gradient Boosting	Challenger	Strong nonlinear modelling	More tuning complexity	Performance vs complexity

Table 5

Phase	Activities	Deliverable	Control
A — Audit	Schema, quality, target and leakage checks	Dataset profile	No modelling before audit
B — Split	Lock final test set	Split record	Test set hidden from tuning
C — Baseline	Run simple reference	Baseline metrics	Fixed protocol
D — Candidate evaluation	CV for all models	Fold metrics	Same folds/settings
E — Tuning	Controlled parameter search	Tuning log	Development data only
F — Selection	Compare metrics and stability	Selected pipeline	Predefined rule
G — Error analysis	Confusion + case inspection	Error report	Root-cause hypotheses
H — Final test	One-time final evaluation	Final metrics	Test remains untouched until now
I — Documentation	Record decisions and limitations	Experiment report	Reproducibility check

Table 6

Risk	Detection	Impact	Mitigation	Contingency	Verification
Class imbalance	Target-frequency audit	Minority recall can collapse	Stratification, macro F1, class weights where justified	Acquire representative data or document restricted scope	Per-class recall/support
Data leakage	Feature-timing audit; suspiciously high CV score	Inflated performance	Fold-safe pipelines; remove post-outcome fields	Remove/redefine feature and rerun	Controlled before/after CV
Overfitting	Large train-CV gap	Poor generalization	Depth/regularization controls; CV tuning	Simplify model/features	Train vs CV gap
Small sample	High CV variance; low class support	Unstable conclusions	Report mean $\pm$ SD; reduce complexity	Repeated CV if statistically reasonable	Fold distribution
Distribution shift	Subgroup/time performance changes	Weak field performance	Time/group-aware validation where possible	Limit deployment scope; monitor drift	Subgroup metrics
Missing data	Null-rate audit	Biased/unreliable predictions	Pipeline-based imputation; missingness logging	Drop unreliable feature or manual-review rule	Before/after quality report
Noisy measurements	Impossible/extreme values	Unstable model behaviour	Range checks and documented cleaning	Exclude sensor/feature if unreliable	Data-quality summary
Label ambiguity	Conflicting/uncertain labels	Unreliable target	Document label source and rules	Exclude ambiguous cases from primary analysis	Label audit count

Risk	Detection	Impact	Mitigation	Contingency	Verification
Test contamination	Test predictions used during tuning	Optimistic final estimate	Lock test set	Fresh test/evaluation set if contaminated	Experiment log
Reproducibility failure	Different runs give unexplained results	Cannot validate findings	Seed, version, config logging	Re-run from clean environment	Reproducibility checklist

Table 7

Model	Parameters to explore	Reason
Logistic Regression	C, regularization type where supported	Controls complexity
Decision Tree	max_depth, min_samples_split, min_samples_leaf	Controls overfitting
Random Forest	n_estimators, max_depth, min_samples_leaf, max_features	Controls ensemble size and tree complexity
Gradient Boosting	n_estimators, learning_rate, max_depth/min_samples controls	Balances fit and complexity

Table 8

Criterion	Required evidence	Outcome
Data integrity	Verified schema, target and leakage review	Pass / Rework
Pipeline integrity	Fold-safe preprocessing	Pass / Rework
Model quality	Macro F1 + class-wise recall vs baseline	Pass / Rework
Stability	CV mean and variability	Pass / Investigate
Error understanding	Major errors categorized and investigated	Pass / Investigate
Final integrity	Test set untouched until final evaluation	Pass / Invalid if violated
Reproducibility	Configuration and code version recorded	Pass / Rework

Table 9

Day	Primary work	Deliverable	Risk control
1	Acquire/verify data; audit schema and labels	Dataset profile	Leakage/quality gate
2	Create locked test split and preprocessing	Fold-safe pipeline	Test isolation
3	Run baseline and candidate CV	Comparison table	Same protocol
4	Tune promising candidates	Tuning log	Development data only
5	Perform error analysis	Confusion/error report	Root-cause review
6	Final refit and one-time test	Final evaluation	Test lock
7	Documentation and reproducibility review	Final report + experiment log	Rerun checklist